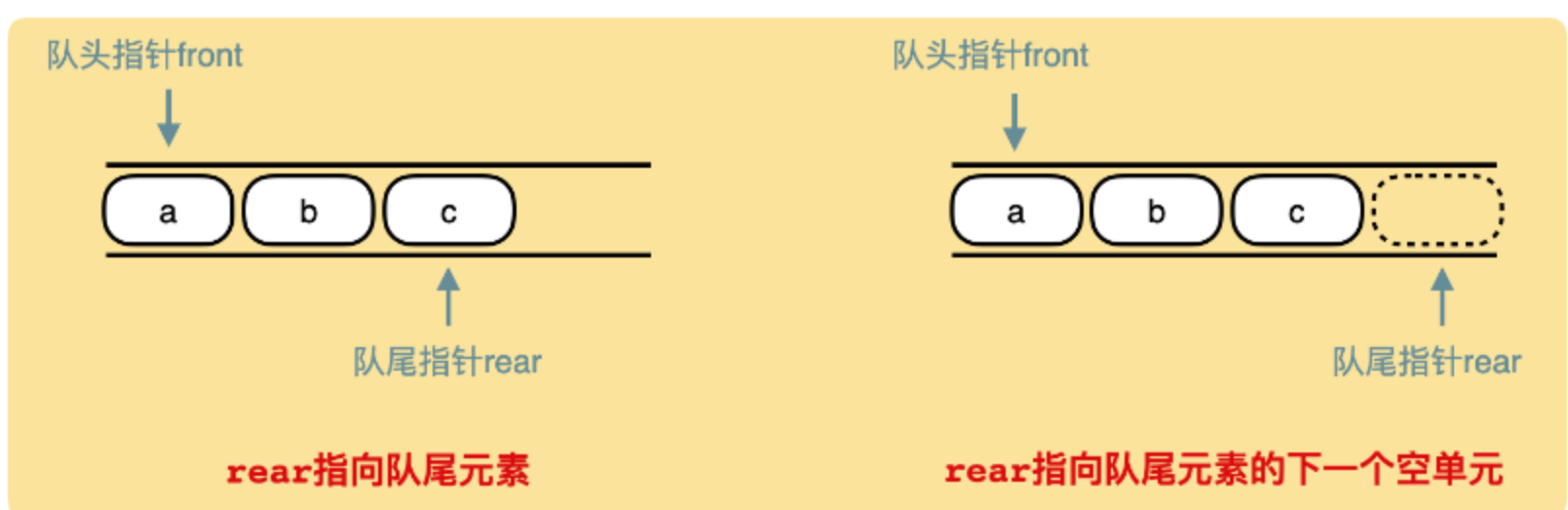


队列

1.队列的定义和基本操作

- 定义
 - 只能在一端进行插入，另一端进行删除的线性表
 - 先进先出
 - 对于同一个入队序列，如果我们规定的操作序列不同，那么出队序列也会不同，所以，在没有限定操作序列的前提下，入队序列和出队序列会呈现一对多的关系
- 基本操作
 - InitQueue(&Q) —— 初始化队列，构造一个空队列Q
 - QueueEmpty(Q) —— 判队列空；若队列Q为空，返回true，否则返回false
 - EnQueue(&Q,x) —— 入队操作；若队列Q未满，将x加入队尾
 - DeQueue(&Q,&x) —— 出队操作；若队列Q非空，删除队首元素，并通过x返回其值
 - GetHead(Q,&x) —— 读队首元素但不出队；若队列Q非空，将队首元素赋值给x



2.队列的顺序表示

- 顺序队列的定义

```
#define MaxSize 50 //定义顺序队列的最大容量为50
typedef struct{
    ElemType data[MaxSize]; //用数组data存放队列中的元素
    int front, rear; //front和rear分别是队头指针和队尾指针
}SqQueue; //顺序队列的结构类型为SqQueue
```
- 顺序队列的假溢出问题
- 循环队列判队空/队满
 - 方案1：牺牲一个空单元不放元素
 - 队空：Q.front == Q.rear
 - 队满：(Q.rear + 1) % MAXQSIZE == Q.front
 - 方案2：另设标志位
 - 队空：tag == 0, Q.front == Q.rear
 - 队满：tag == 1, Q.front == Q.rear
 - 方案3：另设num变量记录队列中元素个数
 - 队空：num == 0, Q.front == Q.rear
 - 队满：num == MAXQSIZE, Q.front == Q.rear

循环队列的基本操作

- 初始化

```
void InitQueue(SqQueue &Q){
    Q.rear = Q.front = 0;
}
```
- 判队空

```
bool QueueEmpty(SqQueue Q){
    if(Q.rear == Q.front)
        return true;
    else
        return false;
}
```
- 入队

```
bool EnQueue(SqQueue &Q, ElemType x){
    if((Q.rear+1)%MaxSize == Q.front)
        return false;
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear+1)%MaxSize;
    return true;
}
```
- 出队

```
bool DeQueue(SqQueue &Q, ElemType &x){
    if(Q.rear == Q.front)
        return false;
    x = Q.data[Q.front];
    Q.front = (Q.front+1)%MaxSize;
    return true;
}
```

3.队列的链式表示

- 链队列的定义

```
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
typedef struct{
    LinkNode *front, *rear;
}LinkQueue;
```
- 链队列的基本操作
 - 初始化

```
void InitQueue(LinkQueue &Q){
    Q.front = Q.rear = (LinkNode*)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
```
 - 判队空

```
bool QueueEmpty(LinkQueue Q){
    if(Q.front == Q.rear)
        return true;
    else
        return false;
}
```
 - 入队

```
void EnQueue(LinkQueue &Q, ElemType x){
    LinkNode *s = (LinkNode*)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;
    Q.rear = s;
}
```
 - 出队

```
bool DeQueue(LinkQueue &Q, ElemType &x){
    if(Q.front == Q.rear)
        return false;
    LinkNode *p = Q.front->next;
    x = p->data;
    Q.front->next = p->next;
    if(Q.rear==p)
        Q.rear = Q.front;
    free(p);
    return true;
}
```

4.双端队列

允许在两端进行插入和删除的线性表